

ENGR 102 Summer Bridge

Day 12 Instructor Key

For Loops, Range, Counters, and Accumulators

20 points • approximately 20-25 minutes

Name		UIN		Section	
------	--	-----	--	---------	--

Boolean-operator convention. Python operators appear as [and](#), [or](#), and [not](#). Their lowercase spelling is required in code.

Show intermediate state for trace credit. Program credit requires one coherent solution. Unless a prompt says otherwise, give exact Python 3 output.

Question 1. Trace the complete loop state.

5 points

```
total = 1
count = 0
for value in range(3, 10, 2):
    total = total * 2 + value
    if total % 3 == 0:
        count = count + 1
print(total, count)
```

Show value, total, and count after every pass, then give exact output.

Answer and explanation

range gives 3, 5, 7, 9. States: after 3, total 5 and count 0; after 5, total 15 and count 1; after 7, total 37 and count 1; after 9, total 83 and count 1. Exact output: 83 1.

Question 2. Diagnose and repair an inclusive-bound bug.

3 points

```
product = 1
for factor in range(2, 5):
    product = product * factor
print(product)
```

The intent is $2 * 3 * 4 * 5$. Give actual, expected, cause, repair, and one test.

Answer and explanation

Actual is 24; expected is 120. `range(2, 5)` excludes 5. Repair with `range(2, 6)`. Test the smallest endpoint case by changing the desired interval to 2 through 2: the correct loop must execute once and produce 2.

Question 3. Program construction**12 points**

Write one complete Python program that satisfies every requirement.

- Read days as a positive integer and target as a float.
- If days is not positive, print invalid.
- Otherwise read exactly days floating-point usage values, compute their total, and count values strictly above target.
- Print total and above_count on separate lines. Include one boundary test for usage equal to target.

Answer and explanation

```
days = int(input())
target = float(input())

if days <= 0:
    print("invalid")
else:
    total = 0.0
    above_count = 0
    for day in range(days):
        usage = float(input())
        total = total + usage
        if usage > target:
            above_count = above_count + 1
    print(total)
    print(above_count)
```

The guard prevents an invalid repetition count. total and above_count are initialized once; usage is read inside the loop exactly days times.

Boundary test: days 2, target 5, usages 5 and 6 prints total 11.0 and above_count 1. Equality must not increment a strictly-above counter.

Scoring guidance

2 guard/input, 2 initialization, 3 exact loop and input placement, 2 total/count logic, 1 output, 2 meaningful equality test and expected result.